Ls LFU cache → O(1)

Ls least Frequency Used.

Ls size = 2

put(1,10) ✓

put(2,20) ✓

get(1) → 10

put(3,30) → del L.R.U

get(2) → -1

get(3) → 30

put(4,4) → fil

get(1) → -1

get(3) → 30

get(4)

get(key) → gets the value of key if exists, else -1.

put(key, value)

Ls update the value of key if present, or insert the key if not present.

Ls when the cache is full, it removes the LFU one if tie, then the least of them.

→ freq → for this eg = cap = 2

(→ (1,10) (2,30) (3,30) (4,4)

get(1) put(3,30) → size > 2

get(3) → get(4)

LRU in LRU's

2→ (1,10) (3,30) (4,4)

put(4,4) tie → get(3)

3 → (3,30)

Size = 3

when put

1 → (4,40) (3,30) (2,25) (1,10)

z y x

now empty

2→ (4,40) (2,25) (3,30)

now change in freq

(→ (5,50)

3→ (2,25)

→ DLL

↳ map< freq, list> freq list;

↳ map< key, node*> key node.

capacity = 3

→ now inserting (5,50)

→ when 1 is empty.

freq = 1 → tracks least freq guy as of now.

(4, -)
(3, -)
(2, y)
(1, x)

(key, node)

→ put(1,10) put(2,20) put(3,30) put(4,40) get(3) get(2) get(4)

exceeds capacity

30 20 40

put(5,50) put(2,25)